Fortune Placement & Architecture

How Fortunery deals its fortunes - what types exist, where each one appears in the game flow, and how the pre-test and post-test are kept balanced and comparable.

Fortunery (fortune-cookie) · game version with Part 1 / Part 2 stages · source of truth: `src/config/fortuneBlueprint.js`, `src/utils/fortunePlan.js`, `src/data/fortunePool.js`

---

1. Purpose and research design
2. Fortune taxonomy
3. Architecture
4. Fortune placement in the game flow
5. Tray test layout (pre-test / post-test)
6. The level rounds
7. How a plan is dealt
8. The fortune pool
9. Scoring, unlocking and progress
10. Analytics
11. Configuration reference

1. Purpose and research design

Fortunery measures how well a player recognises phishing links. Every fortune is a short message (context) containing a URL. The game is played in two **parts**, unlocked like stages. Each part has a disguised **pre-test** (the "Training" trays), a teaching **level**, and a disguised **post-test** (the "Final" trays). Comparing a player's post-test with their pre-test shows what they learnt in that part.

Two rules make that comparison valid:

- **Same shape, different fortunes.** The pre-test and post-test of a part have the identical layout and the identical mix of fortune types and URL attacks - but never share a fortune.
- **Everything is covered, equally.** Every test contains every fortune type the part teaches, and every kind of phishy URL (URL attack).

2. Fortune taxonomy

2.1 Fortune types

A fortune pairs a URL (legitimate or suspicious) with context that either matches the URL's brand or doesn't. There are four types - two faulty, two valid:

Type (key)	URL	Context	Verdict	Example
`sus_related` Suspicious, related	Suspicious	About the brand the URL imitates - it invites you to use the fake link	**Faulty**	"...new album on https://www.youtyube.com/music"
`legit_related` Legitimate, related	Legitimate	About that same brand	Valid	"...trailer is now streaming on https://www.youtube.com"
`legit_unrelated` Legitimate, unrelated	Legitimate	About something the brand has nothing to do with	**Faulty**	"Make your payments safer with https://www.youtube.com"
`sus_warned` Suspicious, warned	Suspicious	Warns the reader away from the fake link	Valid	"Never sign in on links like https://www.youtubbe.com"

The underlined words are the fortune's `invalidPart` - what the player should highlight when marking. Only faulty types have one.

2.2 URL attacks (kinds of phishy URL)

Every suspicious URL (both `sus_` types) is tagged with how it imitates the real brand:

Attack (key)	How it fakes the brand	Example
Typosquatting `typosquatting`	The brand name misspelt	youtyube.com, netflx.com
Combosquatting `combosquatting`	The real brand plus extra words	amazon-gift-rewards.com
Homoglyph `homoglyph`	Look-alike characters	arnazon.com (rn for m), paypa1.com
TLD swap `tld_swap`	The right name on the wrong ending	netflix.top, instagram.xyz
Subdomain spoof `subdomain_spoof`	The brand put in front of someone else's domain	amazon.com.prime-renew.co
IP address `ip_address`	A raw number instead of a name	http://192.0.2.17/hdfcbank/login

2.3 Which types each part uses

Part	Teaching level	Fortune types
Part 1 - Spot the fakes	Level 2 (5 domes)	sus_related legit_related legit_unrelated
Part 2 - Read the warnings	Level 3 (6 domes)	sus_related legit_related legit_unrelated sus_warned

Level 1 is no longer part of either part, but remains in the code base.

3. Architecture

Three files define the fortunes; one function deals them; the game shares the result with every screen that shows a fortune; every answer flows to analytics.

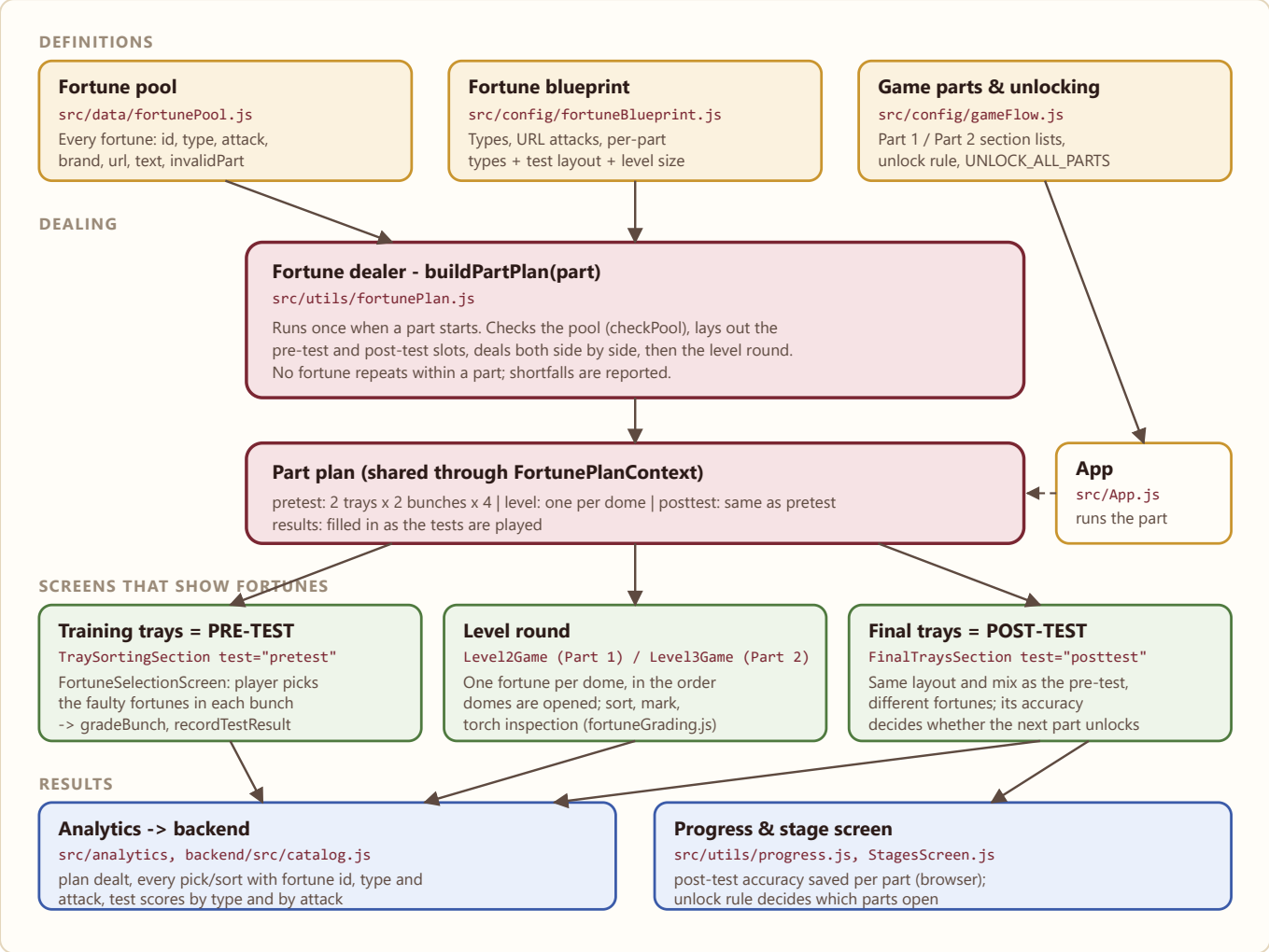


Figure 1 - Architecture. Yellow: definitions you edit. Red: the dealer and the plan it produces. Green: screens that show fortunes. Blue: where the answers go.

3.1 Responsibilities

File	Responsibility
`src/data/fortunePool.js`	The fortunes themselves (currently a 68-fortune starter set: 18 sus_related, 18 sus_warned, 16 legit_related, 16 legit_unrelated). Replace or extend with the real pool.
`src/config/fortuneBlueprint.js`	The four types, the six URL attacks, and per part: which types, the test layout, the level size.
`src/utils/fortunePlan.js`	`checkPool` (validates entries), `buildPartPlan` (deals a part), `gradeBunch`, `recordTestResult`, `planSummary`, `FortunePlanContext`.
`src/config/gameFlow.js`	The parts (`GAME_PARTS`, `PART_ORDER`), the unlock rule and the testing switch `UNLOCK_ALL_PARTS`.
`src/App.js`	Runs one part at a time; builds the part's plan when it starts and provides it to every section; shows the stage screen between parts.
`src/sections/TraySortingSection.js`	The tray minigame for both tests: shows each bunch from the plan, grades it, records the test score.
`src/components/Level2Game.js`, `Level3Game.js`	Deal one plan fortune per dome as it is opened.
`src/utils/fortuneGrading.js`	Grades sorting and marking against `isPhishy` / `invalidPart`.
`src/utils/progress.js`, `StagesScreen.js`	Saved progress and the stage (part select) screen.

4. Fortune placement in the game flow

Each part is a list of sections played in order. Fortunes appear in exactly three places per part - the pre-test, the level round and the post-test - all dealt from one plan at the moment the part starts.

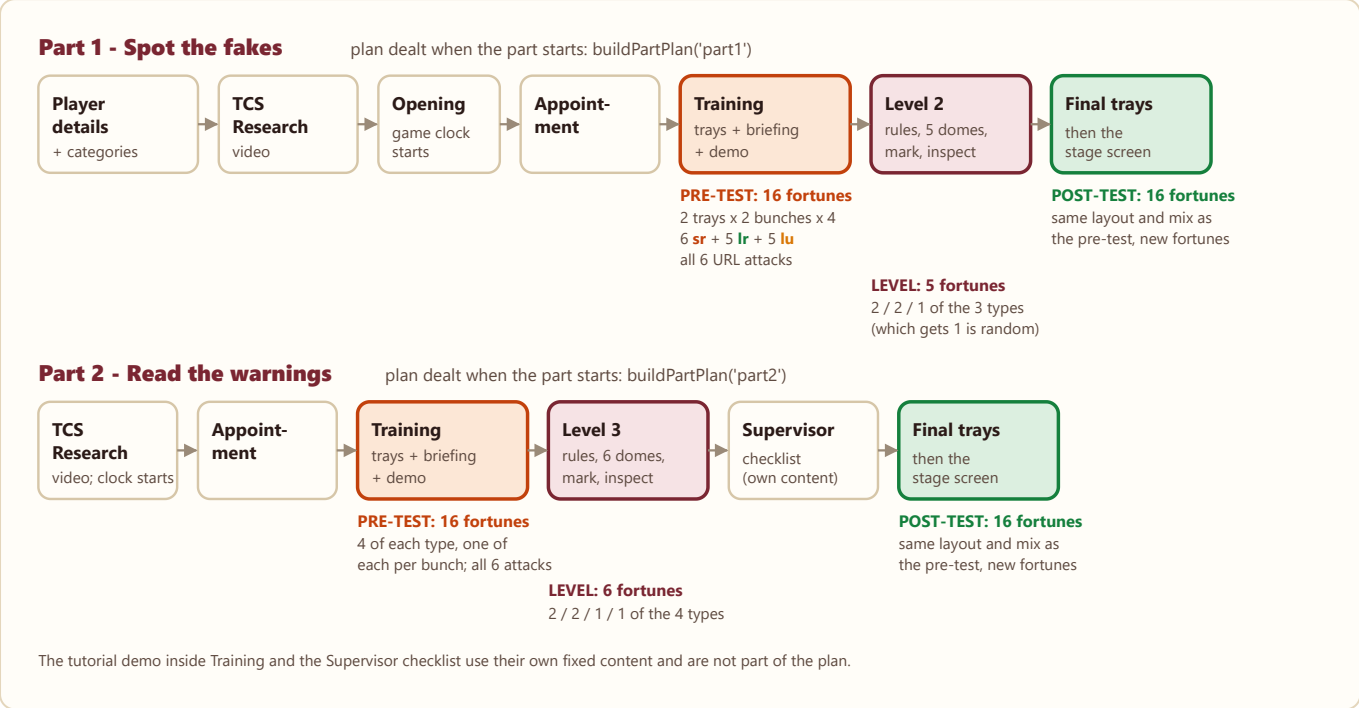


Figure 2 - Where fortunes appear. Orange: pre-test. Red: level round. Green: post-test. Grey sections show no plan fortunes. sr = sus_related, lr = legit_related, lu = legit_unrelated.

4.1 Fortunes per part

Fortune type	Part 1			Part 2		
	Pre-test	Level 2	Post-test	Pre-test	Level 3	Post-test
sus_related	6	1-2	6	4	1-2	4
legit_related	5	1-2	5	4	1-2	4
legit_unrelated	5	1-2	5	4	1-2	4
sus_warned	-	-	-	4	1-2	4
Total	**16**	**5**	**16**	**16**	**6**	**16**
URL attacks covered	all 6, once each	cycled	all 6, once each	all 6 across 8 suspicious	cycled	all 6 across 8 suspicious
Faulty : valid	11 : 5	varies	11 : 5	8 : 8	varies	8 : 8

> **Why Part 1 is 6 / 5 / 5.** 16 fortunes don't divide evenly by 3 types. The spare fortune goes to the suspicious type, because covering all six URL attacks in every test needs at least six suspicious fortunes - and Part 1 has only one suspicious type. The mix is still identical between Part 1's pre-test and post-test.

5. Tray test layout (pre-test / post-test)

Both tests of both parts: **2 trays x 2 bunches per tray x 4 fortunes per bunch = 16 fortunes**. In each bunch the player picks every fortune they think is faulty. The bunch the player clicks first in a tray gets that tray's first set of fortunes, whichever cookie bunch they click.

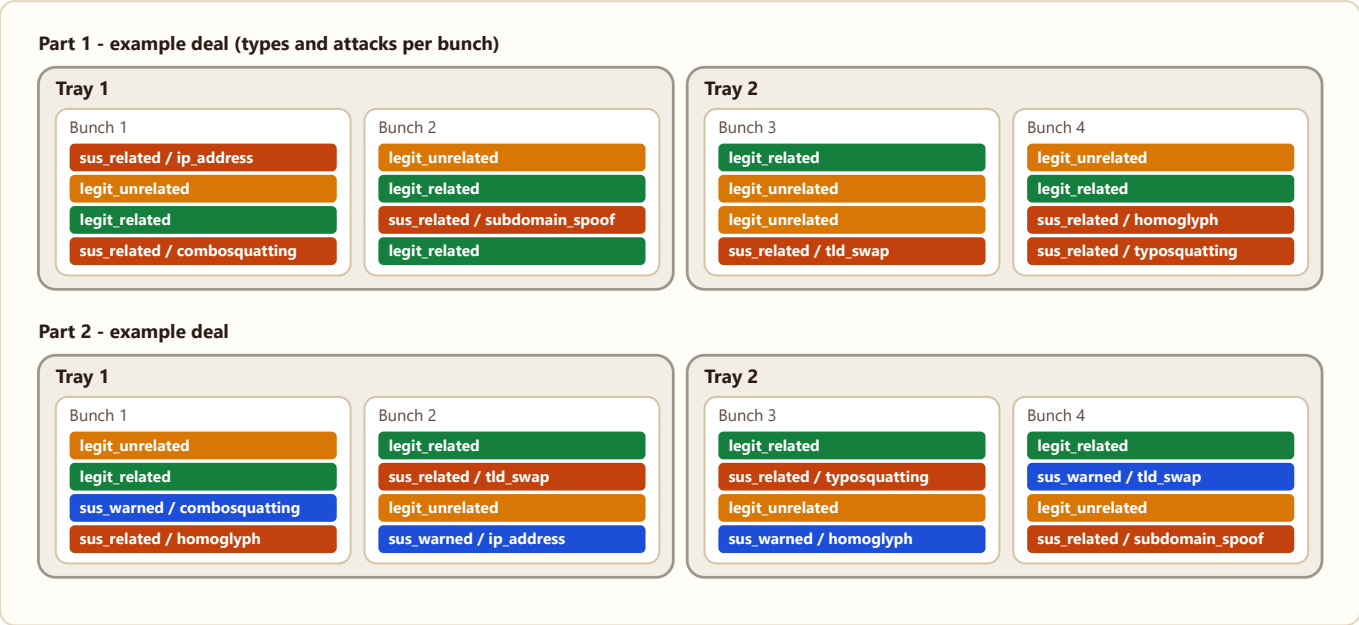


Figure 3 - Two real deals produced by the dealer. Every bunch holds every type of its part; every test covers all six URL attacks; no bunch repeats an attack. Order within a bunch is shuffled.

5.1 Guarantees for every test

- **Size:** 2 trays x 2 bunches x 4 fortunes.
- **Type mix:** equal shares per type; spares go to suspicious types first. Identical in the pre-test and post-test of a part.
- **Every bunch** contains at least one fortune of every type in its part, so each bunch always has something faulty and something valid.
- **URL attacks:** all six appear in every test; within a type, no attack repeats until that type has had them all; no bunch has the same attack twice.
- **No repeats:** no fortune appears twice within a part (pre-test, level and post-test together); a bunch avoids repeating a brand.
- **Fair draw:** pre-test and post-test are dealt slot by slot side by side (random which goes first), so neither gets the leftovers; which fortunes land in which test is random per player.

These were verified over 300 random deals per part with the starter pool: no shortfalls, no repeats, all guarantees held.

6. The level rounds

- **Part 1 - Level 2 (5 domes):** the three types dealt round-robin from a random starting type (so two types get 2 domes and one gets 1, randomly), then shuffled.
- **Part 2 - Level 3 (6 domes):** the four types round-robin from a random start (2 / 2 / 1 / 1), shuffled.
- Suspicious fortunes in the level cycle through a fresh shuffle of the URL attacks.
- The level is dealt after both tests, from the fortunes the tests didn't use.

- Fortunes are handed out in the order the player opens the domes; sorting, marking and the torch inspection are graded against each fortune's type and `invalidPart`.

7. How a plan is dealt

`buildPartPlan(part)` runs once when a part starts (from the stage screen, or straight into Part 1 on a first visit). Steps:

1. **Validate the pool** (`checkPool`): unique ids, known types, attacks on suspicious fortunes, an `invalidPart` on faulty fortunes that appears in the text, none on valid ones. Problems are logged in development.
2. **Count types per test**: `16 / number of types` each; any remainder goes to suspicious types first.
3. **Lay out the slots**: types dealt round-robin into a line (a type is skipped once used up) and cut into bunches of 4 - so every bunch gets every type before any repeats.
4. **Assign URL attacks**: one shuffled cycle of the six attacks runs through all the test's suspicious slots, type after type; reshuffled if a bunch would repeat an attack.
5. **Deal the tests side by side**: for each slot, the pre-test and post-test each draw a fortune of the slot's type and attack (random order of which draws first), avoiding fortunes already used and brands already in the bunch.
6. **Deal the level** from what's left (section 6).
7. **Fall back gracefully** if the pool runs short: drop the brand rule, then the attack, then (last resort) repeat a fortune. Every fallback is recorded in `plan.shortfalls`, logged in development and sent to analytics.
8. **Shuffle** the order inside each bunch.

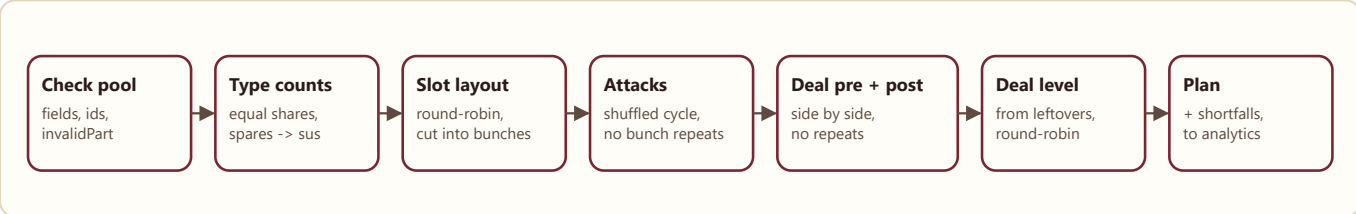


Figure 4 - The dealing pipeline inside buildPartPlan.

8. The fortune pool

8.1 Format

One object per fortune in `src/data/fortunePool.js` :

```
{
  id:          'sr-typo-youtube',        // unique, never reused - analytics records it
  type:        'sus_related',            // sus_related | legit_related | legit_unrelated | sus_warned
  attack:      'typosquatting',          // sus_ types only (see section 2.2); omit for legit_
  brand:       'YouTube',                // the brand the URL is, or pretends to be
  url:         'https://www.youtyube.com/music',   // exactly as it appears in the text
  text:        'Your favourite artist has just dropped a new album on https://www.youtyube.com/music',
  invalidPart: 'youtyube'                // faulty types only: the exact words to highlight
}
```

Type	`attack`	`invalidPart` should be
sus_related	required	the fake part of the URL (e.g. `youtyube`, `.top`, `gift-rewards`, the IP)
legit_unrelated	omit	the context that doesn't fit the brand (e.g. `make your payments safer`)
legit_related	omit	omit
sus_warned	required	omit (the fortune is valid)

8.2 How many fortunes are needed

Per part	Minimum without repeats	Recommended
Each type in the part	pre-test share + post-test share + its level share (Part 1: up to 14 sus_related; about 12 of the others)	14+ per type, more for variety between players
Each sus_ type, per URL attack	2 (one for each test)	3+

The starter set has 68 fortunes: 18 sus_related and 18 sus_warned (3 per URL attack each), 16 legit_related and 16 legit_unrelated. Plans are dealt per part, so a fortune can appear in Part 1 and again in Part 2; it never repeats within a part.

> IP addresses in the starter pool use the ranges reserved for documentation (192.0.2.x, 198.51.100.x, 203.0.113.x), so they can never point at a real server. Keep to these when writing new IP-address fortunes.

9. Scoring, unlocking and progress

- **Tray tests:** a fortune is answered correctly when the player picks it as faulty exactly when it is faulty. Test accuracy = correct / 16. Scores are broken down by type and by URL attack.
- **Level rounds:** sorting into Faulty / Approved, marking the `invalidPart`, and the torch inspection are graded per fortune (`src/utils/fortuneGrading.js`).
- **Unlocking:** Part 1 is always open. Part 2 opens when Part 1 has been completed with **at least 70% post-test accuracy** (`unlock.minPosttestAccuracy`).
- **Testing switch:** `UNLOCK_ALL_PARTS = true` opens every part regardless of score (currently on). Set it to `false` for real players.
- **Progress** (completed parts, best post-test accuracy, the player's country for currency) is saved in the browser (`localStorage`, key `fortunery-progress`). A returning player who has more than one part open starts on the stage screen.
- Part 2 has no title screen, so the game clock starts when Part 2 begins.

10. Analytics

Event	When / what it records
`part_start`, `part_complete`	A part begins / ends; completion carries pre-test and post-test accuracy.
`fortune_plan_built`	The exact fortune ids dealt to this player: pre-test and post-test by tray and bunch, the level in dome order, and any shortfalls.
`tray_fortunes_selected`	One bunch answered: test (pretest/posttest), what was shown (id, type, attack, faulty), selected ids, correct count, decision time.
`tray_test_complete`	A whole test: answered, correct, accuracy, and accuracy by type and by URL attack.
`fortune_revealed`, `fortune_sorted`, `fortune_timed_out`	Level dome events, now with `fortune_id`, `fortune_type` and `url_attack`.

All events are listed in `backend/src/catalog.js`. Pre-test vs post-test improvement per player = `part_complete.posttest_accuracy - pretest_accuracy`, or per type / attack from the two `tray_test_complete` events.

11. Configuration reference

To change...	Edit
The fortunes	`FORTUNE_POOL` in `src/data/fortunePool.js`
Which types a part uses	`PART_BLUEPRINTS.partN.types` in `src/config/fortuneBlueprint.js`
Test size (trays, fortunes per bunch)	`PART_BLUEPRINTS.partN.test` (currently `{ trays: 2, fortunesPerBunch: 4 }`)
Level size	`PART_BLUEPRINTS.partN.level.domes` (must match the level's board: 5 for Level 2, 6 for Level 3)
The URL attack list	`URL_ATTACKS` - a test needs at least as many suspicious fortunes as attacks to cover them all
Sections in each part	`GAME_PARTS` in `src/config/gameFlow.js`
The unlock threshold	`GAME_PARTS.part2.unlock.minPosttestAccuracy`
Open every part for testing	`UNLOCK_ALL_PARTS` in `src/config/gameFlow.js`